

# Laboratório de Concorrência · Aula 10

Gabriel de Souza Carvalho · RA 24027655-2

Gerado em 09/10/2026, 20:14:35 · tempo registrado no cronômetro: 95:20

## Preparação: Concorrência em cinco minutos

---

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

Cooperação é quando uma tarefa depende da outra, então ela tem que esperar a outra terminar uma parte antes de seguir. Competição é quando as duas querem usar a mesma coisa, que só dá para uma usar por vez.

Cooperação: numa lanchonete o garçom só leva o lanche depois que a cozinha termina de fazer.

Competição: duas pessoas querendo usar o mesmo banheiro, uma tem que esperar a outra sair.

## Prática 1: A condição de corrida

---

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

2.000.000, já que são duas threads somando 1 milhão cada. Mas como as duas mexem no mesmo contador, acho que pode dar menos.

**1ª execução (n = 1.000.000)**

1099287

**2ª execução (n = 1.000.000)**

1347737

**3ª execução (n = 1.000.000)**

1386844

**Saída do programa**

OneCompiler

Upgrade

AI

Java

Run

952 ms

Save

Login

Main.java

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

I/O

AI Agent

952 ms

esperado: 2000000

obtido: 1099287

Ready

Terminal

Light

Wiki

Internet

OneCompiler

Upgrade

AI

Java

Run

904 ms

Save

Login

Main.java

```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

I/O

AI Agent

904 ms

esperado: 2000000

obtido: 1347737

Ready

Terminal

Light

Wiki

Internet

OneCompiler

Upgrade

AI

Java

Run

967 ms

Save

Login

Main.java

+

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)  
2 // Cole em <https://onecompiler.com/java>  
3 public class Main {  
4 static int contador = 0; // variável COMPARTILHADA  
5  
6 public static void main(String[] args) throws InterruptedException {  
7 int n = 1\_000\_000; // repetições de cada thread  
8 Runnable incrementa = () -> {  
9 for (int i = 0; i < n; i++) contador++;  
10 };  
11 Thread t1 = new Thread(incrementa);  
12 Thread t2 = new Thread(incrementa);  
13 t1.start(); t2.start(); // as duas executam ao mesmo tempo  
14 t1.join(); t2.join(); // espera as duas terminarem  
15 System.out.println("esperado: " + 2 \* n);  
16 System.out.println("obtido: " + contador);  
17 }  
18 }

Console

I/O

AI Agent

esperado: 2000000  
obtido: 1386844

Ready

Terminal

Light

Wiki

Internet

OneCompiler

Upgrade

AI

Java

Run

939 ms

Save

Login

Main.java

+

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)  
2 // Cole em <https://onecompiler.com/java>  
3 public class Main {  
4 static int contador = 0; // variável COMPARTILHADA  
5  
6 public static void main(String[] args) throws InterruptedException {  
7 int n = 1000; // repetições de cada thread  
8 Runnable incrementa = () -> {  
9 for (int i = 0; i < n; i++) contador++;  
10 };  
11 Thread t1 = new Thread(incrementa);  
12 Thread t2 = new Thread(incrementa);  
13 t1.start(); t2.start(); // as duas executam ao mesmo tempo  
14 t1.join(); t2.join(); // espera as duas terminarem  
15 System.out.println("esperado: " + 2 \* n);  
16 System.out.println("obtido: " + contador);  
17 }  
18 }

Console

I/O

AI Agent

esperado: 2000  
obtido: 2000

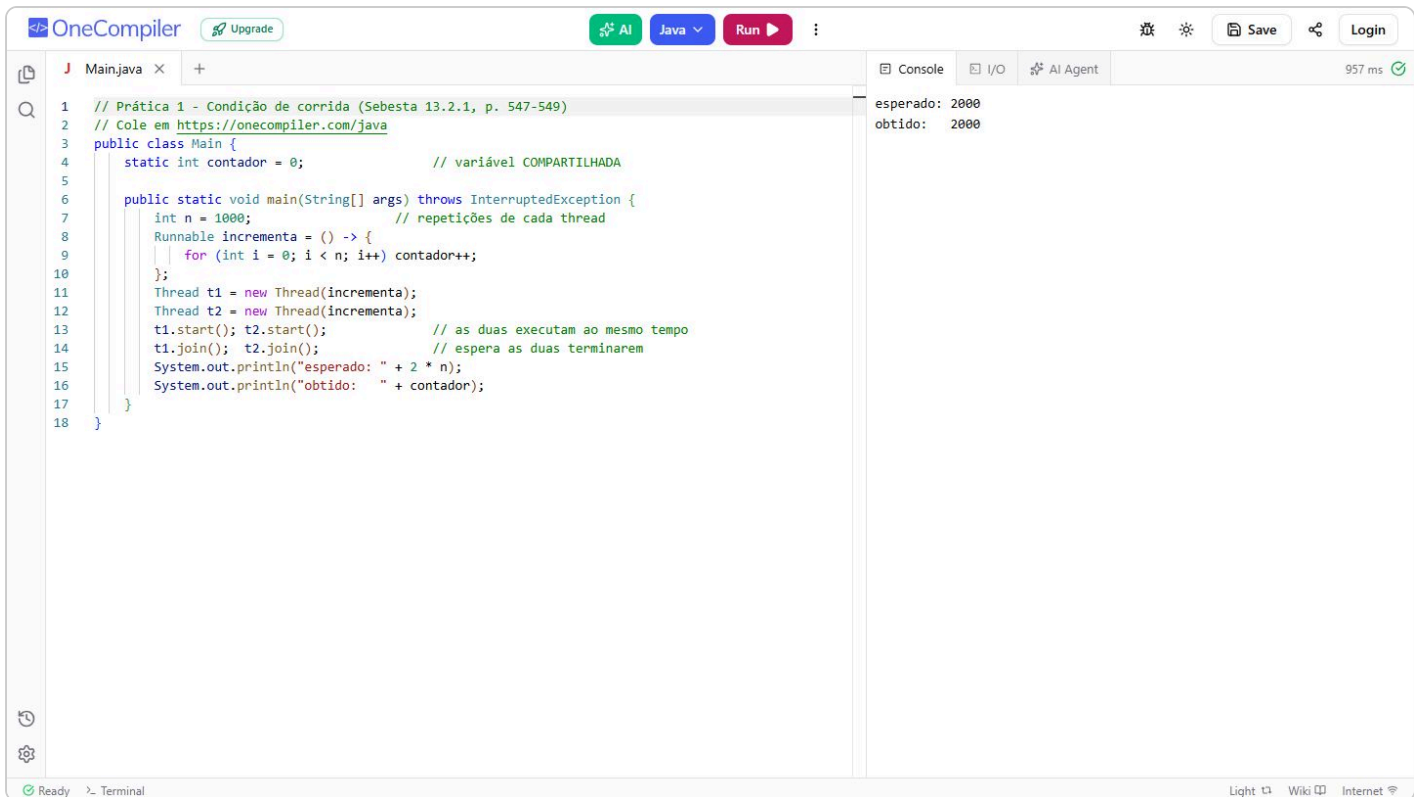
Ready

Terminal

Light

Wiki

Internet



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

esperado: 2000  
obtido: 2000

### 1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram. Deu 1.099.287, 1.347.737 e 1.386.844, sempre bem menos que 2 milhões e cada vez um valor diferente. Muda porque quem decide qual thread roda em cada momento é o escalonador, e isso nunca acontece igual. Cada vez que as duas threads se cruzam no meio do contador++ uma soma se perde, e quantas vezes isso acontece varia de uma execução pra outra.

### 1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Imaginando o contador valendo 10:

1. A lê 10
2. antes de A gravar, B também lê 10
3. A soma 1 e fica com 11, B também soma e fica com 11
4. A grava 11
5. B grava 11 por cima

Teve duas somas mas o contador só foi de 10 pra 11. A soma da A foi apagada pela B.

### 1.3 Com $n = 1000$ , o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com  $n = 1000$  as duas execuções deram 2000, certinho. Como são poucas repetições, uma thread termina antes da outra começar, então elas quase não se cruzam. O perigo é que o erro continua lá, só fica raro. Nos testes, com pouca coisa acontecendo, o programa passa e parece certo, e aí em produção, com muita gente usando ao mesmo tempo, começa a dar valor errado de vez em quando e ninguém consegue reproduzir o erro.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

### Ordens testadas no simulador

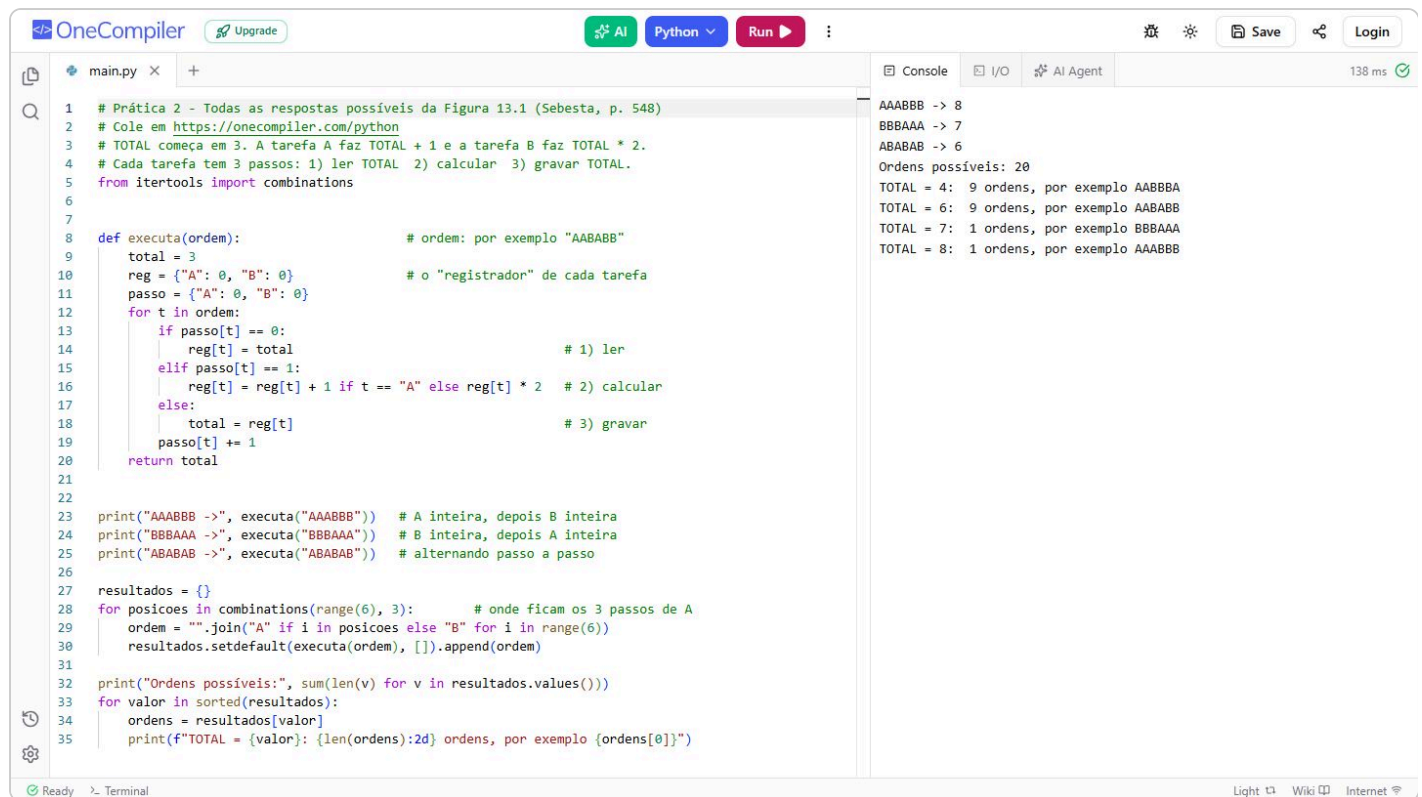
AAABBB → 8

BBBAAA → 7

ABABAB → 6

AABBBA → 4

## Saída do programa



```
1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB"))
24 print("BBBAAA ->", executa("BBBAAA"))
25 print("ABABAB ->", executa("ABABAB"))
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
```

## 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis e aparecem 4 resultados diferentes: 4, 6, 7 e 8. Só 1 ordem dá 8 (AAABBB). O 7 também sai de só 1 ordem (BBBAAA), e o 4 e o 6 saem de 9 ordens cada. Então o resultado "certo" é justamente o mais difícil de sair.

## 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem AABBBA:

1. A lê TOTAL = 3
2. A calcula  $3 + 1 = 4$  (só no registrador)
3. B lê TOTAL, que ainda é 3
4. B calcula  $3 \times 2 = 6$
5. B grava 6
6. A grava 4 por cima

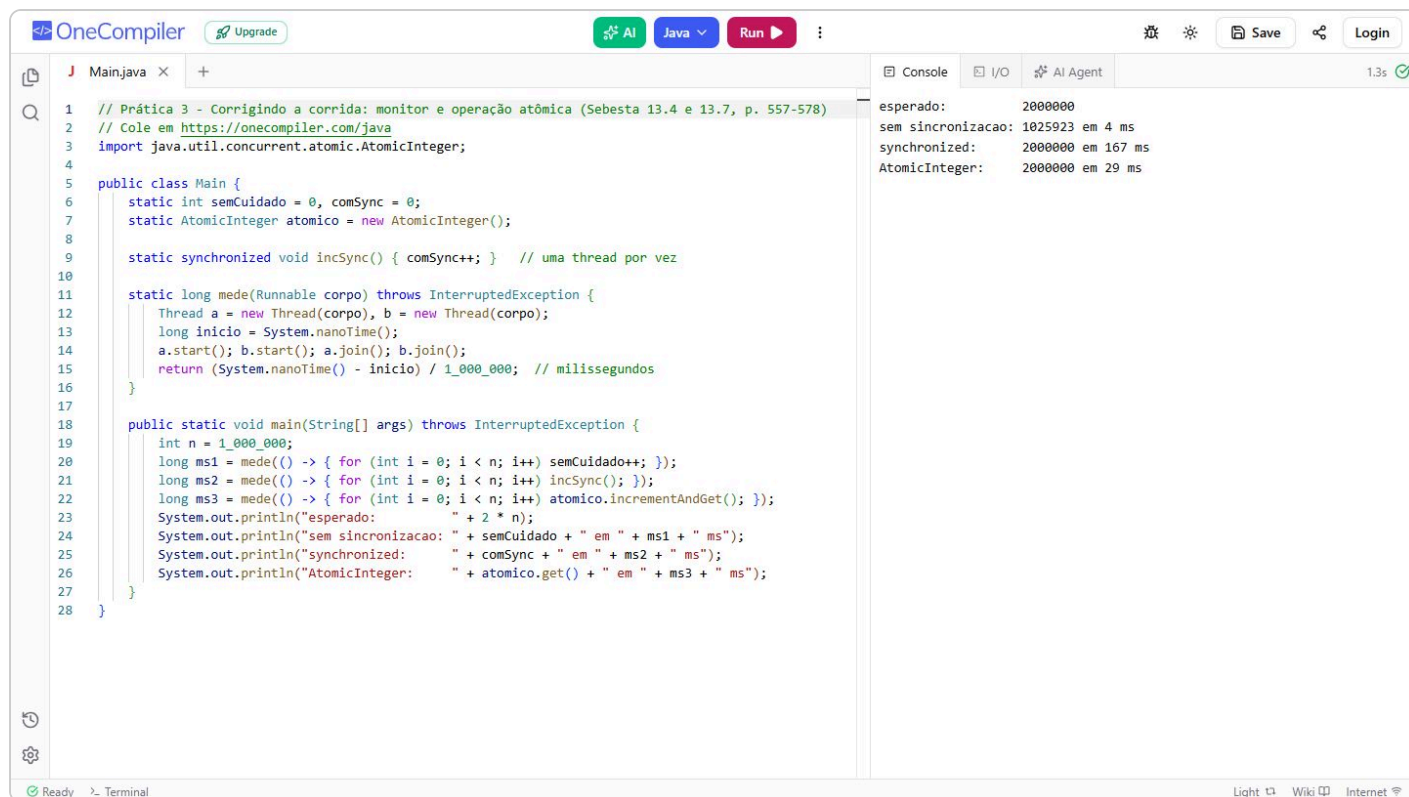
Fica 4. A operação da B se perdeu porque a A gravou um valor que ela calculou com o TOTAL antigo e apagou o 6.

## Prática 3: Corrigindo a corrida

**Previsão:** As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a sem sincronização vai ser a mais rápida, mas vai dar errado igual na prática 1. Entre as outras duas, chuto que o AtomicInteger é mais rápido.

## Saída do programa

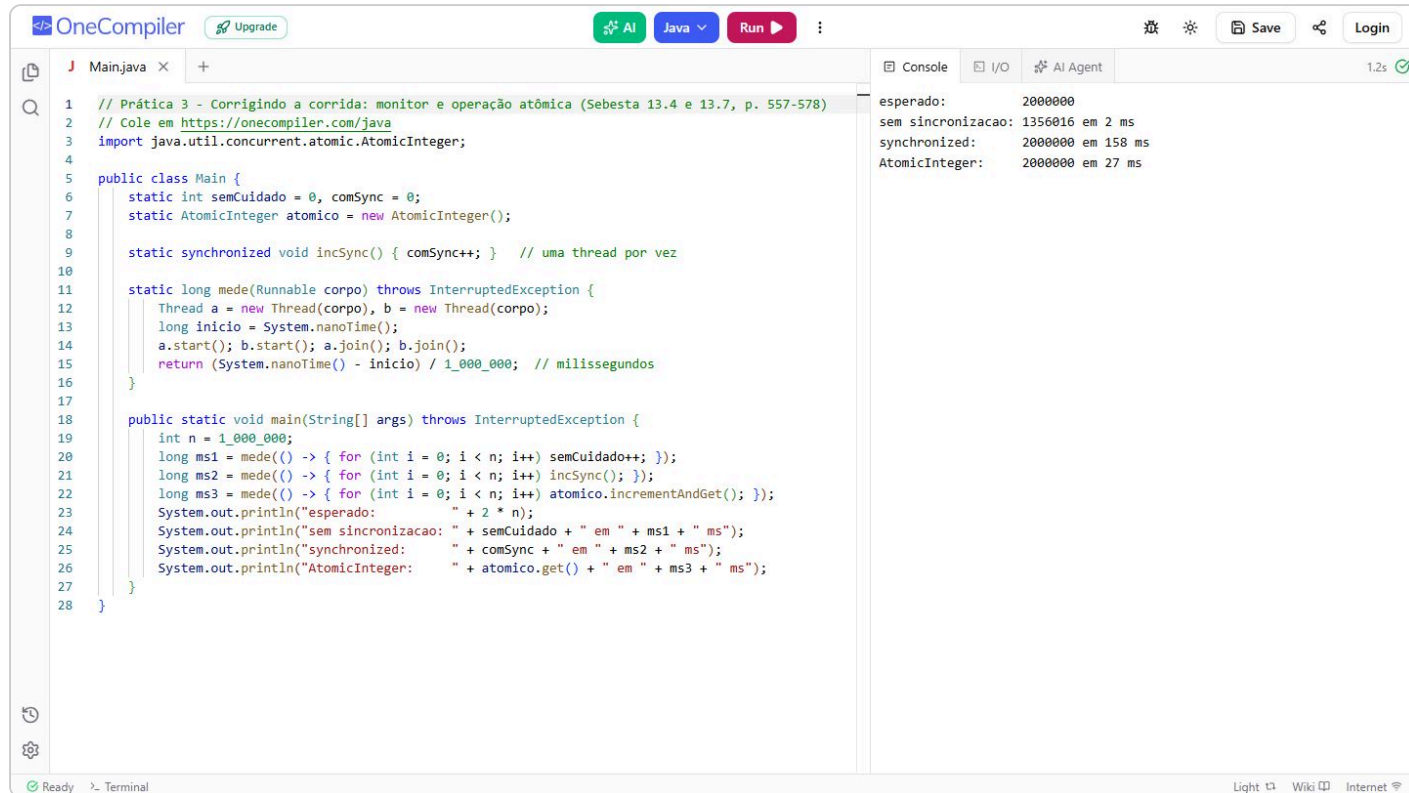


The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a test comparing three synchronization methods: a semaphore (semCuidado), a synchronized block (incSync), and an AtomicInteger (atomico). The test runs 1,000,000 iterations for each. The console output shows the expected value is 2,000,000. The semaphore method took 102,592,3 ms. The synchronized method took 200,000 ms. The AtomicInteger method took 200,000 ms.

```
1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }
```

Console Output:

```
esperado: 2000000
sem sincronizacao: 1025923 em 4 ms
synchronized: 200000 em 167 ms
AtomicInteger: 200000 em 29 ms
```



The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a test comparing three synchronization methods: a semaphore (semCuidado), a synchronized block (incSync), and an AtomicInteger (atomico). The test runs 1,000,000 iterations for each. The console output shows the expected value is 2,000,000. The semaphore method took 135,601,6 ms. The synchronized method took 200,000 ms. The AtomicInteger method took 200,000 ms.

```
1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }
```

Console Output:

```
esperado: 2000000
sem sincronizacao: 1356016 em 2 ms
synchronized: 200000 em 158 ms
AtomicInteger: 200000 em 27 ms
```

**3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?**

Deram certo o `synchronized` e o `AtomicInteger` (2.000.000). A sem sincronização deu 1.025.923 e 1.356.016. Nos tempos, a sem sincronização levou 2 a 4 ms, o `AtomicInteger` uns 28 ms e o `synchronized` uns 160 ms, bem mais lento. As versões certas demoram mais porque têm que garantir que só uma thread mexe no contador por vez. No `synchronized` cada incremento pega e solta a trava, e enquanto uma thread está com ela a outra fica parada esperando. No `AtomicInteger` o processador faz a operação de uma vez só, sem travar a thread, por isso foi bem mais rápido que o `synchronized`, mas ainda é mais lento que não fazer controle nenhum.

### 3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um `AtomicInteger` em cada saldo resolve? Por quê? O que seria preciso?

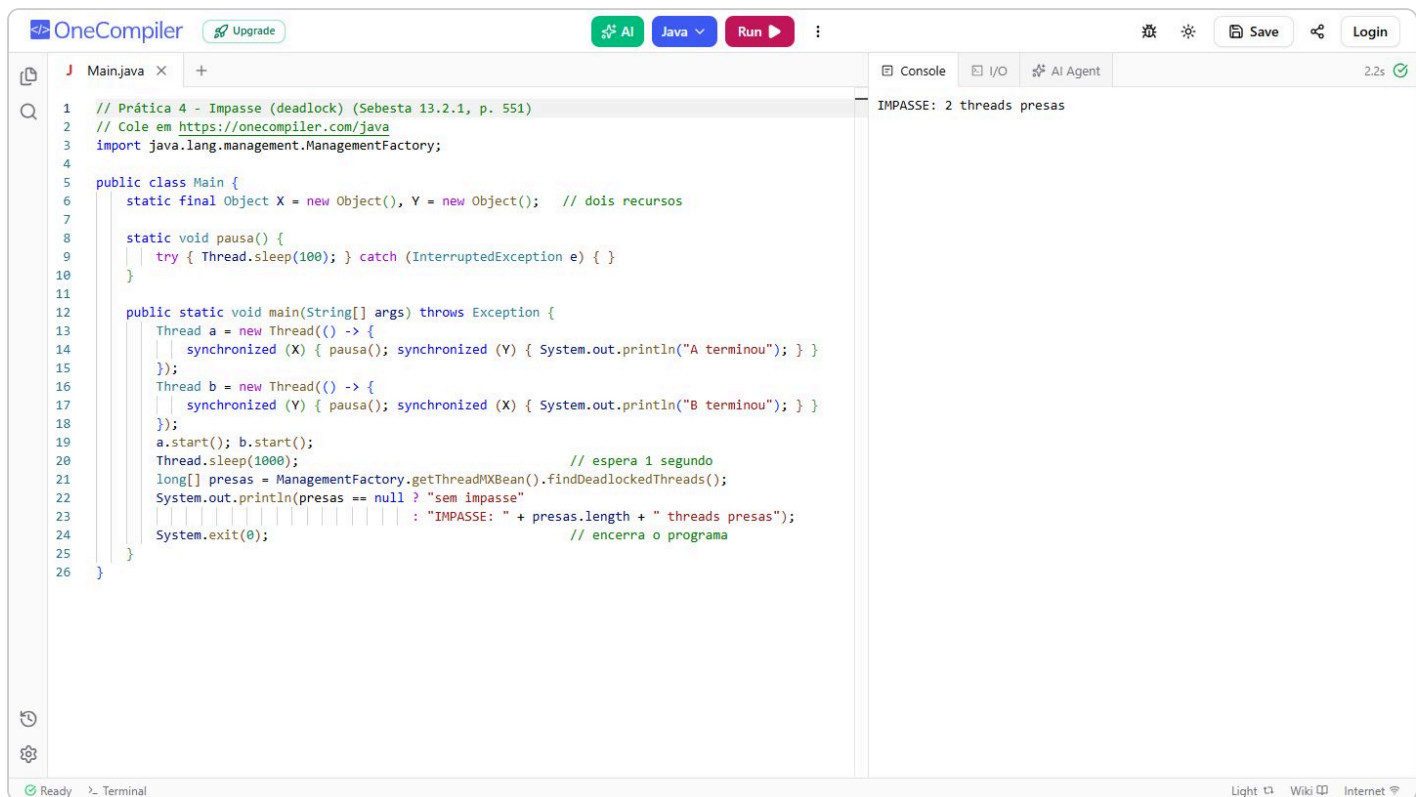
Não resolve. O `AtomicInteger` garante cada operação sozinha, mas a transferência são duas operações (tirar de uma conta e colocar na outra) que têm que acontecer juntas. No meio do caminho outra thread pode ver o dinheiro sumido, ou duas transferências podem ver saldo suficiente ao mesmo tempo e as duas tirarem, deixando a conta negativa. Precisaria travar as duas contas durante a transferência toda (sempre na mesma ordem, por causa do impasse) ou fazer isso numa transação do banco de dados.

## Prática 4: Impasse (deadlock)

**Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?**

Acho que não. A vai ficar com X esperando Y, e B com Y esperando X, então as duas travam.

**Saídas (antes e depois da correção)**



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) {}
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

IMPASSE: 2 threads presas

```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

Console output:  
A terminou  
B terminou  
sem impasse

#### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A pegou X e ficou esperando Y. B pegou Y e ficou esperando X. Cada uma está segurando o que a outra precisa e nenhuma solta, então ficam esperando uma pela outra pra sempre. Por isso não apareceu nem "A terminou" nem "B terminou".

#### 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se as duas pegam X primeiro, só uma consegue X e a outra fica esperando sem segurar nada. Aí a primeira pega Y, termina, solta tudo e a segunda continua. Não tem como uma ficar esperando a outra em círculo. Exemplo: duas transferências ao mesmo tempo, uma de João pra Maria e outra de Maria pra João, cada uma travando primeiro a conta de origem. Dá pra resolver travando sempre primeiro a conta de número menor.

## Prática 5: Mensagens em vez de memória compartilhada

**Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?**

Com 3 trabalhadores cada um faz 2 tarefas, então uns 200 ms. Com 1, uma de cada vez, 600 ms. Com 6, tudo junto, uns 100 ms.

#### Tempo com 1 trabalhador

600ms

#### Tempo com 3 trabalhadores

200ms

#### Tempo com 6 trabalhadores

100ms

## Saída do programa

OneCompiler Upgrade

AI Go Run

Save Login

main.go

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

Console

trabalhador 2 fez a tarefa 3  
trabalhador 3 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 2 fez a tarefa 4  
trabalhador 1 fez a tarefa 6  
trabalhador 3 fez a tarefa 5  
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

Ready Light Wiki Internet

OneCompiler Upgrade

AI Go Run

Save Login

main.go

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

Console

trabalhador 1 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 1 fez a tarefa 3  
trabalhador 1 fez a tarefa 4  
trabalhador 1 fez a tarefa 5  
trabalhador 1 fez a tarefa 6  
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

Ready Light Wiki Internet

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

trabalhador 5 fez a tarefa 6  
trabalhador 2 fez a tarefa 3  
trabalhador 6 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 3 fez a tarefa 4  
trabalhador 4 fez a tarefa 5  
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

365 ms

## 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador ele faz as 6 tarefas uma depois da outra,  $6 \times 100 = 600$  ms. Com 3, cada um faz 2 ao mesmo tempo que os outros, 200 ms. Com 6, cada um faz uma e todas rodam juntas, 100 ms. Não precisa de synchronized porque os trabalhadores não mexem em nenhuma variável em comum. Cada tarefa chega pelo canal, e o canal garante que cada uma vai pra um trabalhador só.

## Saídas (antes e depois da troca)

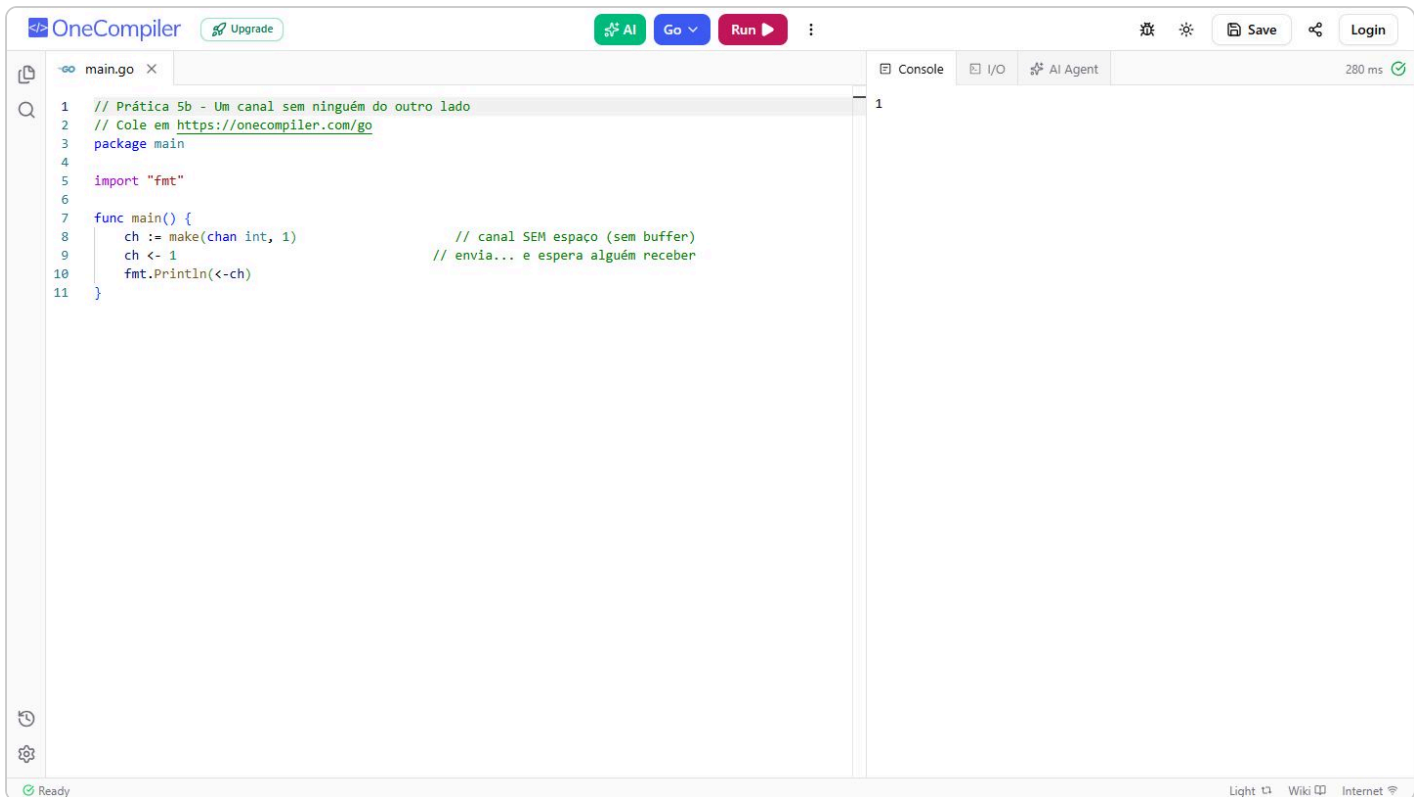
```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int) // canal SEM espaço (sem buffer)
9     ch <- 1 // envia... e espera alguém receber
10     fmt.Println(<-ch)
11 }
```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:  
main.main()  
/box/main.go:9 +0x38  
exit status 2

Debug this error

Exit 1



```
1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1)           // canal SEM espaço (sem buffer)
9     ch <- 1                          // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }
```

1

Ready Light Wiki Internet

## 5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem espaço, quem envia fica esperando alguém receber. Só tem a main, e ela travou no envio, então nunca chega na linha do `<-ch`. O Go vê que ninguém mais pode andar e encerra com deadlock. Com espaço para 1, o 1 fica guardado no canal e o envio não precisa esperar ninguém. Na linha seguinte a própria main recebe e imprime 1.

## Prática 6: O compilador como guarda

**Previsão:** Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que nem compila. O Rust não deve deixar a thread mexer na variável da main assim.

**Saídas (o erro e o resultado com move)**

OneCompiler

Upgrade

AI Rust Run

Exit 1

Save Login

HelloWorld.rs

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

Console I/O AI Agent

```
error[E0373]: closure may outlive the current function, but it borrows
`contador`, which is owned by the current function
--> HelloWorld.rs:7:27
|
7 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value `contador`
8 |         contador += 1; // a thread altera a
|         variável de main
|         ----- `contador` is borrowed here
|
note: function requires argument type to outlive ``static``
--> HelloWorld.rs:7:13
|
7 |     let t = thread::spawn(|| {
|         ^
8 |         contador += 1; // a thread altera
|         a variável de main
9 |     });
|         ^
help: to force the closure to take ownership of `contador` (and any
other referenced variables), use the `move` keyword
|
7 |     let t = thread::spawn(move || {
|                             ++++

error[E0502]: cannot borrow `contador` as immutable because it is also
borrowed as mutable
--> HelloWorld.rs:11:20
|
7 |     let t = thread::spawn(|| {
|         -
|         -- mutable borrow occurs here
|         |
|         |
```

Upgrade

AI
Rust
Run

Save
Exit 1

HelloWorld.rs

```

1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1;           // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

Console

```

help: to force the closure to take ownership of `contador` (and any
other referenced variables), use the `move` keyword
|
7 |     let t = thread::spawn(move || {
|                               ++++

error[E0502]: cannot borrow `contador` as immutable because it is also
borrowed as mutable
--> HelloWorld.rs:11:20
|
7 |         let t = thread::spawn(|| {
|                               - -- mutable borrow occurs here
|
8 |             contador += 1;           // a thread altera
a variável de main
| |             ----- first borrow occurs due to use of `contador`
in closure
9 |         });
|         |_____- argument requires that `contador` is borrowed for
`static`
10 |         t.join().unwrap();
11 |         println!("{}", contador);
|                               ^^^^^^^^^ immutable borrow occurs here

note: requirement that the value outlives `static` introduced here
-->
/rustc/48a229ceafd4985c50990b14116b6d856af0985/library/std/src/thread/f

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.
```

Ready

Light
Wiki
Internet

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

warning: value captured by `contador` is never read  
--> HelloWorld.rs:8:9

```
8 |         contador += 1; // a thread altera a
  |         ^^^^^^^^^
  |
  = help: did you mean to capture by reference instead?
  = note: `[warn(unused_assignments)]` (part of `[warn(unused)]`) on
by default
```

warning: value assigned to `contador` is never read  
--> HelloWorld.rs:8:9

```
8 |         contador += 1; // a thread altera a
  |         ^^^^^^^^^
  |
  = help: maybe it is overwritten before being read?
```

warning: 2 warnings emitted

0

## Saída do programa

```
1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10        let c = Arc::clone(&contador);
11        threads.push(thread::spawn(move || {
12            for _ in 0..100_000 {
13                *c.lock().unwrap() += 1; // trava, soma e destrava
14            }
15        }));
16    }
17    for t in threads { t.join().unwrap(); }
18    println!("contador = {}", *contador.lock().unwrap());
19 }
```

contador = 400000

### 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move a thread leva uma cópia do contador, não o contador da main. Ela soma 1 na cópia dela, e o da main continua 0, que é o que aparece no println. O próprio compilador avisa nos warnings que o valor capturado nunca é lido. Para compartilhar de verdade tem que usar Arc com Mutex, como na 6b, que deu 400000.

## 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

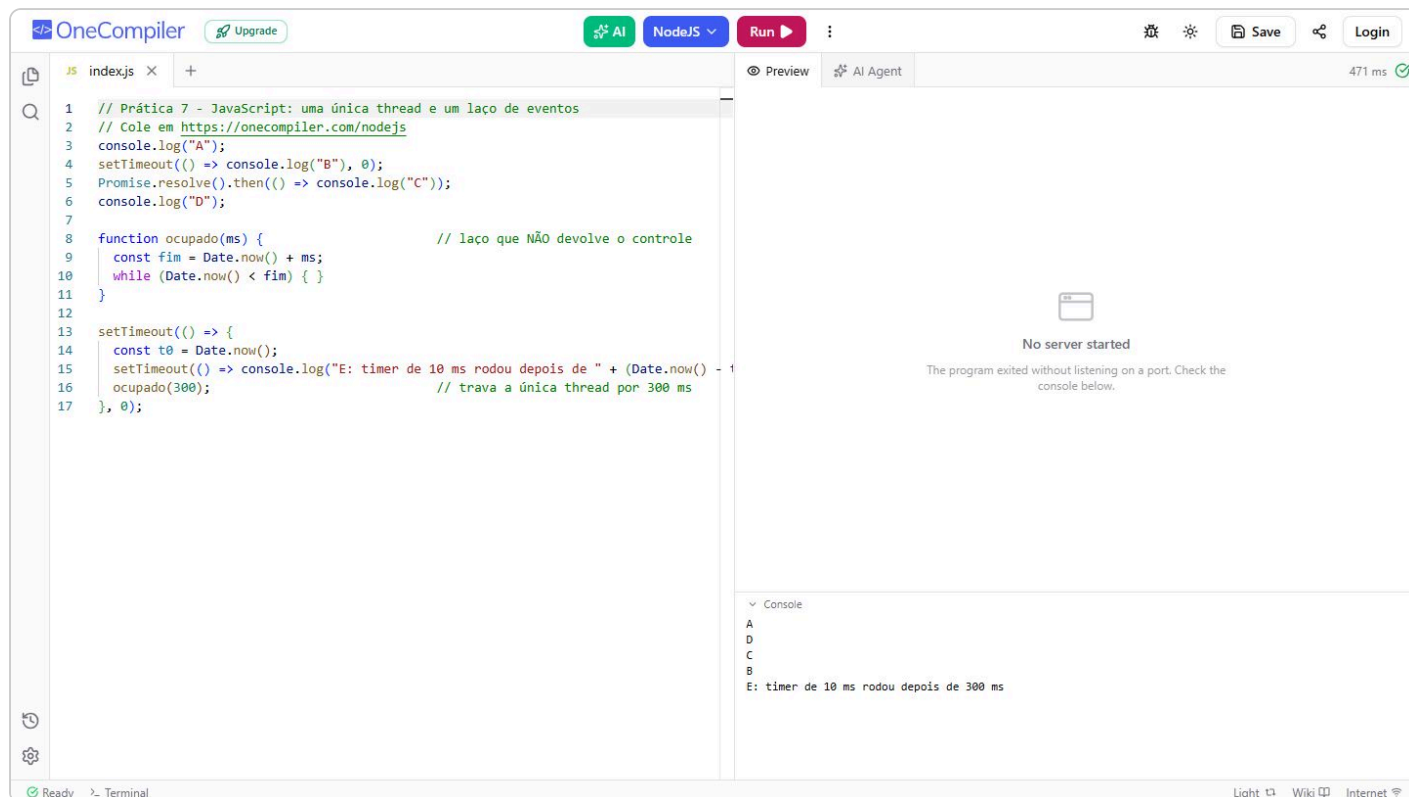
No Java o erro só apareceu rodando: compilou normal e deu valor errado, e com  $n = 1000$  até deu certo, então poderia passar nos testes. No Rust o erro apareceu na compilação, o programa nem chegou a rodar. A vantagem é que esse tipo de erro não chega em produção. O custo é que é mais difícil de programar: tem que entender posse, empréstimo, move, Arc, Mutex e ficar ajustando até o compilador aceitar.

## Prática 7: Uma thread só: o laço de eventos

### Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A e D primeiro, porque são normais. Depois acho que vem C e por último B, mesmo o B tendo tempo 0.

### Saída do programa



```
1 // Prática 7 - JavaScript: uma única thread e um laço de eventos
2 // Cole em https://onecompiler.com/nodejs
3 console.log("A");
4 setTimeout(() => console.log("B"), 0);
5 Promise.resolve().then(() => console.log("C"));
6 console.log("D");
7
8 function ocupado(ms) { // laço que NÃO devolve o controle
9   const fim = Date.now() + ms;
10  while (Date.now() < fim) { }
11 }
12
13 setTimeout(() => {
14   const t0 = Date.now();
15   setTimeout(() => console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0)), 10);
16   ocupado(300); // trava a única thread por 300 ms
17 }, 0);
```

No server started  
The program exited without listening on a port. Check the console below.

Console

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

### 7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. A e D rodam direto. O `setTimeout`, mesmo com 0, não roda na hora: ele vai pra uma fila e só executa depois que o código atual acaba. A promessa (C) vai pra fila de microtarefas, que roda antes da fila dos timers. Por isso C sai antes de B, e os dois depois de D.

### 7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque a função `ocupado(300)` prendeu a única thread por 300 ms. O timer de 10 ms ficou pronto, mas só pode rodar quando a thread fica livre, então rodou depois de uns 300 ms. Num servidor Node isso quer dizer que um cálculo pesado trava todo mundo: enquanto ele roda nenhum outro usuário é atendido. Coisa pesada tem que ir pra outra thread (worker) ou outro serviço.

## **Fechamento: Quem impede a corrida?**

---

### **Java sem sincronização**

Ninguém: há corrida

### **Java com synchronized**

O programador, com sincronização

### **Go com canais**

O projeto: não há memória compartilhada

### **Rust com Arc e Mutex**

O compilador

### **JavaScript**

Só existe uma thread

### **8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?**

Condição de corrida. As duas compras leem que tem 1 lugar livre antes de qualquer uma gravar, e o mesmo lugar acaba vendido duas vezes. Pra evitar, verificar e reservar tem que ser uma operação só. Por exemplo, no banco de dados fazer o UPDATE só se o lugar ainda estiver livre e ver se alguma linha foi alterada. Quem chegar depois recebe "esgotado".

### **8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

Me surpreendeu o mesmo programa em Java dar um resultado diferente cada vez que roda, e às vezes até o certo, enquanto o Rust nem deixa compilar.